

UnifAI / Raksha Enterprise Database Configuration Guide

Complete Persistence Architecture, Optimization, Security & Disaster Recovery

Document Version: 2.4.0

Classification: Enterprise Database Administration & Data Architecture

Target Audience: Database Administrators (DBA), DevOps Engineers, Systems Architects

1. Persistence Architecture & Database Engine Support

UnifAI utilizes GORM as its Object-Relational Mapping (ORM) layer, supporting three primary relational database engines:

Engine	Production Recommended	Minimum Version	Typical Use Case	High Availability Support
PostgreSQL	YES (Preferred)	14.x / 15.x / 16.x	Enterprise production, multi-tenant fleet, high concurrency.	Streaming replication, Patroni, PgBouncer
MySQL / MariaDB	Compatible	8.0+ / 10.6+	Secondary choice for MySQL-exclusive infrastructures.	Group Replication, Galera Cluster
SQLite	NO (Dev only)	3.40+	Local unit tests and developer laptop sandbox (sqlite_static).	Single-process file lock (No clustering)

2. Production PostgreSQL Step-by-Step Installation

2.1 Installing PostgreSQL 16 on Ubuntu 22.04 / 24.04 LTS

Execute the following commands on your database server:

```
# Add official PostgreSQL APT repository
sudo apt update && sudo apt install -y curl ca-certificates gnupg
lsb-release sudo install -d /etc/apt/keyrings curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc
| sudo gpg --dearmor -o /etc/apt/keyrings/postgresql.gpg echo "deb
[signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" | sudo tee /etc/apt/sources.list.d/pgdg.list # Install PostgreSQL 16 server and client tools
sudo apt update sudo apt install -y postgresql-16 postgresql-contrib-16 # Verify installation and service status
sudo systemctl enable postgresql sudo systemctl status postgresql
```

2.2 Provisioning User, Database & Permissions

Log in as the **postgres** system user and execute the provisioning commands:

```
sudo -u postgres psql
```

Execute SQL commands within the **psql** console:

```
-- 1. Create dedicated application user with SCRAM-SHA-256 password hashing CREATE USER unifai_user WITH
PASSWORD 'StrongEnterpriseDBPassword_ReplaceInProd!_2026'; -- 2. Create production database with UTF-8
encoding CREATE DATABASE unifai_db WITH OWNER unifai_user ENCODING 'UTF8' LC_COLLATE 'en_US.UTF-8' LC_CTYPE
'en_US.UTF-8'; -- 3. Connect to the database \c unifai_db -- 4. Grant schema privileges GRANT ALL PRIVILEGES
ON DATABASE unifai_db TO unifai_user; GRANT ALL ON SCHEMA public TO unifai_user; ALTER DEFAULT PRIVILEGES IN
SCHEMA public GRANT ALL ON TABLES TO unifai_user; ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON
SEQUENCES TO unifai_user; -- 5. Enable optional performance and UUID extensions CREATE EXTENSION IF NOT
EXISTS "uuid-oss"; CREATE EXTENSION IF NOT EXISTS "pg_trgm"; CREATE EXTENSION IF NOT EXISTS "btree_gist"; --
Exit psql \q
```

3. Network Authentication & Security Configuration

3.1 pg_hba.conf Authentication Rules

Edit `/etc/postgresql/16/main/pg_hba.conf` to enforce modern **scram-sha-256** password authentication and restrict network access strictly to the application server:

```
# TYPE DATABASE USER ADDRESS METHOD # Local administrative access via UNIX domain socket local all postgres
peer local all all scram-sha-256 # IPv4 local loopback connections (if backend runs on same host) host
unifai_db unifai_user 127.0.0.1/32 scram-sha-256 # Dedicated Application Server Subnet (if backend is on a
separate host/container) host unifai_db unifai_user 172.16.0.0/16 scram-sha-256 host unifai_db unifai_user
10.0.0.0/8 scram-sha-256 # Reject all other unauthorized external traffic host all all 0.0.0.0/0 reject
```

3.2 Binding & Listening Address (postgresql.conf)

Edit `/etc/postgresql/16/main/postgresql.conf`:

```
# Listen only on loopback and internal private VPC interface listen_addresses = 'localhost, 172.16.10.5' port
= 5432 max_connections = 300
```

Reload PostgreSQL to apply configuration changes:

```
sudo systemctl reload postgresql
```

4. Enabling SSL/TLS Encryption for Database Connections

For enterprise compliance, all database traffic across networks must be encrypted in transit.

4.1 Generating PostgreSQL Server Certificates

```
sudo mkdir -p /etc/postgresql/ssl cd /etc/postgresql/ssl # Generate private key for PostgreSQL server sudo
openssl genrsa -out server.key 4096 sudo chmod 600 server.key # Generate self-signed certificate (or issue
via corporate CA) sudo openssl req -new -x509 -days 3650 -key server.key -out server.crt -subj
"/CN=db.internal.yourcompany.com" sudo chmod 644 server.crt sudo chown -R postgres:postgres
/etc/postgresql/ssl
```

4.2 Configuring PostgreSQL for SSL

In `/etc/postgresql/16/main/postgresql.conf`:

```
ssl = on ssl_cert_file = '/etc/postgresql/ssl/server.crt' ssl_key_file = '/etc/postgresql/ssl/server.key'
ssl_min_protocol_version = 'TLSv1.2' ssl_ciphers = 'HIGH:!aNULL:!MD5'
```

4.3 Updating Application `.env` for SSL Mode

In your UnifAI `.env` file, configure:

```
DB_TYPE=postgres DB_HOST=db.internal.yourcompany.com DB_PORT=5432 DB_NAME=unifai_db DB_USER=unifai_user
DB_PASSWORD=StrongEnterpriseDBPassword_ReplaceInProd!_2026 # SSL Modes: 'disable', 'require', 'verify-ca',
or 'verify-full' DB_SSL_MODE=require
```

5. Schema & Table Architecture

UnifAI organizes its database tables into two core logical domains managed via GORM auto-migration:

5.1 ConfigStore Domain (System State & Policies)

- **users**: User profiles, hashed passwords, roles, status (**pending**, **approved**, **disabled**), spending limits.
- **roles**, **permissions**, **resource_catalog**: Granular RBAC definitions (33 resources × 6 operations).
- **access_profiles**: Reusable permission and quota templates.
- **model_providers**: Third-party vendor credentials (encrypted at rest), weights, health status.
- **virtual_keys**: Developer tokens, token budgets, rate limits, allowed model lists.
- **target_websites**: URLs, domains, and regex patterns intercepted by the Browser Guard fleet.
- **guard_rules**: Real-time DLP patterns, regexes, keywords, and AI Guard Bot policies.
- **system_settings**: Global configurations, SMTP settings, branding, security thresholds.

5.2 LogsStore Domain (Observability & Forensics)

- **inference_logs**: Records model ID, input tokens, output tokens, latency, cost, virtual key ID, and user ID.
- **browser_ai_audit_logs**: Records intercepted employee prompts, target websites, matched DLP rules, redaction diffs, and blocking actions.
- **audit_logs**: Mutation logs capturing administrative actions for compliance reporting.

6. PostgreSQL Performance Tuning (**postgresql.conf**)

Tune these parameters based on your database host RAM:

```
# ===== # OPTIMIZED FOR 16 GB RAM
DEDICATED DATABASE SERVER # ===== #
Memory settings shared_buffers = 4GB # 25% of total RAM effective_cache_size = 12GB # 75% of total RAM
maintenance_work_mem = 1GB work_mem = 16MB # Per query sort memory wal_buffers = 16MB # Checkpoints and Write
Ahead Log (WAL) min_wal_size = 1GB max_wal_size = 16GB checkpoint_completion_target = 0.9 checkpoint_timeout
= 15min # Query Planner Optimization random_page_cost = 1.1 # Fast SSD / NVMe storage
effective_io_concurrency = 200 # Concurrent disk IO operations # Background Writer bgwriter_delay = 20ms
bgwriter_lru_maxpages = 100 bgwriter_lru_multiplier = 2.0
```

7. Connection Pooling with PgBouncer

For installations exceeding 500 concurrent users or multiple gateway instances, deploy PgBouncer to eliminate connection establishment overhead:

Install PgBouncer:

```
sudo apt install -y pgbouncer
```

Configure **/etc/pgbouncer/pgbouncer.ini**:

```
[databases] unifai_db = host=127.0.0.1 port=5432 dbname=unifai_db [pgbouncer] logfile =
/var/log/postgresql/pgbouncer.log pidfile = /var/run/postgresql/pgbouncer.pid listen_addr = 127.0.0.1
listen_port = 6432 auth_type = scram-sha-256 auth_file = /etc/pgbouncer/userlist.txt pool_mode = transaction
max_client_conn = 1000 default_pool_size = 50 min_pool_size = 10 reserve_pool_size = 5 reserve_pool_timeout =
5
```

Update your **.env** to point to port **6432**:

```
DB_PORT=6432
```

8. Automated Backup & Disaster Recovery

8.1 On-Demand Backup Commands (pg_dump)

```
# 1. Custom binary compressed backup (Recommended for restores) pg_dump -h 127.0.0.1 -U unifai_user -Fc -b -v
-f "unifai_db_$(date +%Y%m%d_%H%M%S).dump" unifai_db # 2. Plain SQL text backup (Gzip compressed) pg_dump -h
127.0.0.1 -U unifai_user -d unifai_db | gzip > "unifai_db_$(date +%Y%m%d_%H%M%S).sql.gz"
```

8.2 Production Automated Daily Backup Script

Save as **/opt/unifai/scripts/db_backup.sh** and make executable (**chmod +x**):

```
#!/bin/bash # ===== # UnifAI
Production PostgreSQL Automated Daily Backup Script #
===== set -e
BACKUP_DIR="/var/backups/unifai" DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/unifai_db_${DATE}.dump" LOG_FILE="/var/log/unifai_db_backup.log"
RETENTION_DAYS=14 mkdir -p "${BACKUP_DIR}" echo "[$(date)] Starting UnifAI PostgreSQL backup..." >>
"${LOG_FILE}" # Execute pg_dump using credentials from ~/.pgpass or environment export
PGPASSWORD="StrongEnterpriseDBPassword_ReplaceInProd!_2026" pg_dump -h 127.0.0.1 -p 5432 -U unifai_user -Fc
-b -v unifai_db > "${BACKUP_FILE}" 2>> "${LOG_FILE}" if [ $? -eq 0 ]; then echo "[$(date)] Backup successful:
${BACKUP_FILE} ($(du -h ${BACKUP_FILE} | cut -f1))" >> "${LOG_FILE}" else echo "[$(date)] ERROR: Backup
failed!" >> "${LOG_FILE}" exit 1 fi # Clean up backups older than retention window find "${BACKUP_DIR}" -name
"unifai_db_*.dump" -type f -mtime +${RETENTION_DAYS} -delete echo "[$(date)] Cleaned up backups older than
${RETENTION_DAYS} days." >> "${LOG_FILE}"
```

Schedule via Crontab to run every midnight at 02:00 AM:

```
# Add to root crontab (crontab -e) 0 2 * * * /opt/unifai/scripts/db_backup.sh >/dev/null 2>&1
```

8.3 Disaster Recovery: Complete Database Restore Procedure

To restore the database from a backup file:

```
# 1. Terminate any active connections to the database sudo -u postgres psql -c "SELECT
pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname = 'unifai_db' AND pid <> pg_backend_pid();" #
2. Drop and recreate database sudo -u postgres psql -c "DROP DATABASE IF EXISTS unifai_db;" sudo -u postgres
psql -c "CREATE DATABASE unifai_db WITH OWNER unifai_user ENCODING 'UTF8';" # 3. Restore schema and data
using pg_restore sudo -u postgres pg_restore -d unifai_db -v -O -x
"/var/backups/unifai/unifai_db_20261003_120000.dump" # 4. Verify table count and integrity sudo -u postgres
psql -d unifai_db -c "\dt"
```

End of Database Configuration Guide.